DESAFIO PUC-RIO 2026 — SIMULADO 2

■ NÍVEL MÁXIMO 1: Lógica Complexa, OBI Sênior, Algoritmos e Casos de Borda

REGRAS OFICIAIS DO SIMULADO (MODO PROVA)

• **1. Condições Estritas de Prova:** Resolução individual, **sem consulta** a materiais externos, fóruns, documentações ou inteligência artificial.

• **2. Tempo de Execução Recomendado:** 2 horas e 30 minutos para responder às 5 questões.

• **3. Padrão de Entrada e Saída (Python 3):** Suas soluções devem usar apenas input() e print() limpos, sem mensagens explicativas nos prompts (ex.: evite input('Digite:')).

• **4. Precisão da Saída:** A formatação deve ser **rigorosamente idêntica** ao especificado no enunciado (respeitando letras maiúsculas, minúsculas, pontuação e ausência de acentos).

• **5. Obrigatoriedade de Testes Próprios:** Antes de enviar seu código para correção, você DEVE testá-lo com os 4 exemplos da tabela de cada questão e **criar 2 casos de teste próprios** (sendo obrigatoriamente um para casos de borda/limite).

• **6. Escada de Ajuda e Correção:** Envie o código em Python 3 assim que concluir cada questão. Caso haja erro, você receberá pistas progressivas para descobrir a falha por conta própria.

SOBRE O NÍVEL MÁXIMO 1

Este simulado eleva a exigência técnica ao nível de fases finais da OBI Sênior e da prova do Desafio PUC-Rio. As questões abordam simulação complexa de algoritmos, matrizes bidimensionais, ordenação por critérios múltiplos, parsing de expressões e gerenciamento de estruturas de dados. Atente-se aos casos de borda e limites!

Q.	Nome da Questão	Tópicos Chave	Dificuldade
Q1	Submatriz de Soma Máxima com Restrição	Matrizes, Validação, Algoritmo Kadane/For	■■■■■
Q2	Parser e Avaliador de Expressões	Strings, Pilha / Aninhamento de Parênteses	■■■■■
Q3	Agendamento com Penalidades	Algoritmos Gulosos, Ordenação Múltipla, Listas	■■■■■
Q4	Simulador de Torneio Dupla Eliminação	Simulação Avançada, Listas de Controle, Dicionários	■■■■■
Q5	Gerenciador de Cache LRU Simplificado	Estruturas de Dados, Listas, Rastreamento	■■■■■

Siga para as próximas páginas e boa prova!

QUESTÃO 1 — Submatriz de Soma Máxima com Restrição Paridade

Enunciado: Dada uma matriz quadrada *N x N* de inteiros, escreva um programa que encontre a submatriz contígua de tamanho *K x K* (onde 1 ≤ K ≤ N) cuja soma dos elementos seja **estritamente máxima**, sob a seguinte condição: a soma de todos os elementos da submatriz deve ser um número **PAR**. Caso nenhuma submatriz *K x K* resulte em soma par, o programa deve imprimir NENHUMA SUBMATRIZ PAR.

Entrada: A primeira linha contém dois inteiros *N* e *K* separados por espaço. As *N* linhas seguintes contêm *N* inteiros separados por espaço representando a matriz.

Saída: Imprimir a soma máxima par encontrada e, na linha seguinte, a posição inicial da submatriz no formato LINHA X COLUNA Y (usando índices base zero). Em caso de empate na soma máxima, escolha a submatriz que aparece primeiro no percurso por linha (menor linha e depois menor coluna).

#	Entrada (N K e Matriz)	Saída Esperada
1	3 2 1 2 3 4 5 6 7 8 9	28 LINHA 1 COLUNA 1
2	3 2 1 3 5 7 9 11 13 15 17	NENHUMA SUBMATRIZ PAR
3	2 1 3 8 4 7	8 LINHA 0 COLUNA 1
4	4 2 2 4 1 3 6 8 5 7 1 3 2 4 5 7 6 8	26 LINHA 2 COLUNA 2

Antes de programar, pense:

1. Como percorrer todas as possíveis submatrizes de tamanho *K x K* em uma matriz *N x N*?
2. Como acumular a soma dos elementos de cada janela *K x K* e verificar a paridade (soma % 2 == 0)?
3. Como garantir o desempate correto mantendo a primeira ocorrência encontrada?

QUESTÃO 2 — Parser e Avaliador de Expressões Aninhadas

Enunciado: Em linguagens de programação, expressões matemáticas podem conter parênteses aninhados. Escreva um programa que receba uma string contendo uma expressão com inteiros, operadores + e *, e parênteses (e). O programa deve primeiro validar se os parênteses estão **perfeitamente balanceados**. Se estiverem incorretos, imprima SINTAXE INVALIDA. Se estiverem corretos, calcule o resultado final respeitando a prioridade dos parênteses e dos operadores (multiplicação antes de adição).

Entrada: Uma única linha contendo uma string *S* sem espaços.

Saída: Imprimir SINTAXE INVALIDA se o balanceamento falhar; caso contrário, imprimir a palavra RESULTADO: seguida do valor inteiro do cálculo.

#	Entrada (S)	Saída Esperada
1	(2+3)*4	RESULTADO: 20

2	2+3*4	RESULTADO: 14
3	((2+3)*4	SINTAXE INVALIDA
4	((1+2)*(3+4))	RESULTADO: 21

Antes de programar, pense:

1. Qual pilha ou contador você pode usar para checar se cada (tem um) correspondente na ordem certa?
2. Como resolver expressões de dentro para fora quando há parênteses aninhados?
3. A função nativa eval() do Python pode ser usada? (Dica: garanta que a validação de sintaxe seja feita antes de avaliar!).

QUESTÃO 3 — Agendamento com Penalidades e Prazos

Enunciado: Um servidor de processamento possui *N* tarefas a serem executadas sequencialmente. Cada tarefa *i* possui um tempo de execução *T_i* (em minutos) e um prazo limite *D_i*. O servidor inicia no minuto 0. Se a tarefa *i* for concluída no minuto *C_i*, o atraso é max(0, C_i - D_i). Escreva um programa que determine a ordem de execução das tarefas que **minimiza o atraso máximo** entre todas as tarefas. Em caso de empate no atraso, escolha a ordem pelo ID original da tarefa (ordem de entrada).

Entrada: A primeira linha contém um inteiro *N*. As *N* linhas seguintes contêm o ID (string/inteiro), o tempo *T* e o prazo *D* de cada tarefa separados por espaço.

Saída: Na primeira linha, imprimir a ordem dos IDs das tarefas separados por seta -> . Na segunda linha, imprimir ATRASO MAXIMO: X onde X é o atraso máximo acumulado.

#	Entrada (N e Tarefas)	Saída Esperada
1	3 A 2 4 B 1 2 C 3 6	B -> A -> C ATRASO MAXIMO: 0
2	3 T1 5 2 T2 1 3 T3 2 1	T3 -> T1 -> T2 ATRASO MAXIMO: 5
3	2 X 4 3 Y 2 2	Y -> X ATRASO MAXIMO: 3
4	4 P1 2 3 P2 3 4 P3 1 2 P4 2 6	P3 -> P1 -> P2 -> P4 ATRASO MAXIMO: 2

Antes de programar, pense:

1. Qual regra gulosa (ex.: ordenar por menor prazo *EDF - Earliest Deadline First*) minimiza o atraso máximo?

2. Como simular o relógio de conclusão acumulada para calcular o atraso exato de cada tarefa na sequência escolhida?

QUESTÃO 4 — Simulador de Torneio de Eliminação Dupla

Enunciado: Em um e-sport, um torneio com 4 times (identificados de 1 a 4) utiliza o sistema de eliminação simples até definir os finalistas. A chave inicial é: Rodada 1 = (Time 1 vs Time 2) e (Time 3 vs Time 4). Os vencedores avançam para a Final dos Vencedores (Rodada 2). Os perdedores da Rodada 1 jogam entre si na Repescagem. O perdedor da Repescagem é eliminado e fica em 4º lugar. O vencedor da Repescagem joga contra o perdedor da Final dos Vencedores para definir o 3º lugar. Escreva um programa que leia os resultados dos jogos e imprima a classificação final do 1º ao 4º lugar.

Entrada: 4 linhas, cada uma contendo dois inteiros representando os gols/pontos dos confrontos na ordem:
Linha 1: J1 (T1 vs T2)
Linha 2: J2 (T3 vs T4)
Linha 3: J3 (Vencedor J1 vs Vencedor J2)
Linha 4: J4 (Perdedor J1 vs Perdedor J2)

Saída: 4 linhas com a classificação final no formato 1 POSICAO: TIME X, 2 POSICAO: TIME Y, 3 POSICAO: TIME Z, 4 POSICAO: TIME W.

#	Entrada (Gols dos Jogos)	Saída Esperada
1	2 1 3 0 1 2 0 1	1 POSICAO: TIME 3 2 POSICAO: TIME 1 3 POSICAO: TIME 4 4 POSICAO: TIME 2
2	3 0 2 1 2 1 1 2	1 POSICAO: TIME 1 2 POSICAO: TIME 3 3 POSICAO: TIME 4 4 POSICAO: TIME 2
3	1 2 0 1 0 3 2 0	1 POSICAO: TIME 4 2 POSICAO: TIME 2 3 POSICAO: TIME 1 4 POSICAO: TIME 3
4	4 1 1 3 3 2 2 1	1 POSICAO: TIME 1 2 POSICAO: TIME 4 3 POSICAO: TIME 2 4 POSICAO: TIME 3

Antes de programar, pense:

1. Como rastrear quem é o Vencedor e Perdedor de cada partida individualmente?
2. Como montar a árvore de jogos sem se confundir com os IDs dos times?

QUESTÃO 5 — Gerenciador de Cache LRU (Least Recently Used)

Enunciado: Em sistemas operacionais, a memória cache possui capacidade limitada *C*. Quando um elemento é solicitado: se já estiver na cache (**CACHE HIT**), ele é promovido ao status de mais recentemente usado. Se não estiver na cache (**CACHE MISS**): se a cache tiver espaço, ele é inserido; se a cache estiver cheia, o elemento **menos recentemente usado (LRU)** é removido para dar lugar ao novo. Escreva um programa que simule um cache LRU com capacidade *C* recebendo uma sequência de *N* requisições.

Entrada: A primeira linha contém a capacidade *C*. A segunda linha contém o número de requisições *N*. A terceira linha contém *N* inteiros/strings separados por espaço representando os itens solicitados.

Saída: Imprimir o número de HITS e MISSES no formato HITS: X | MISSES: Y e, na linha seguinte, os elementos atualmente na cache do mais recente para o menos recente separados por espaço.

#	Entrada (C, N e Requisições)	Saída Esperada
1	3 6 1 2 3 1 4 2	HITS: 2 | MISSES: 4 ESTADO: 2 4 1
2	2 4 A B C A	HITS: 0 | MISSES: 4 ESTADO: A C
3	4 5 10 20 10 30 40	HITS: 1 | MISSES: 4 ESTADO: 40 30 10 20
4	1 4 5 5 5 5	HITS: 3 | MISSES: 1 ESTADO: 5

Antes de programar, pense:

1. Como a estrutura de lista do Python facilita remover uma chave existente (`.remove()`) e reinseri-la no topo (`.insert(0, elem)`)?

2. O que acontece com o elemento no final da lista quando o tamanho ultrapassa a capacidade *C*?

FIM DO SIMULADO 2 — BOA SORTE!